

LightRaptor: Hierarchical Knowledge-Augmented Retrieval-Augmented Generation with Dynamic Graph Reasoning

Sayan Chowdhury, Sourabh Chandra Mishra

Abstract

We present LightRaptor, a novel offline Retrieval-Augmented Generation (RAG) system that integrates hierarchical document clustering with dynamic knowledge graph construction for enhanced multi-hop reasoning. Our approach combines LightRAG-inspired community detection with RAPTOR-style abstractive summarization, creating a unified framework that operates entirely offline on resource-constrained environments. The system constructs a query-adaptive knowledge graph through incremental entity extraction and co-occurrence analysis, enabling structured reasoning paths that complement traditional semantic retrieval. Evaluated on 2,000 curated scientific Wikipedia articles, LightRaptor achieves 23% improvement in answer quality and 16% gain in factual accuracy over baseline RAG systems, while maintaining sub-10 second query latency on consumer hardware. Our ablation studies demonstrate that structured multi-evidence fusion significantly outperforms naive concatenation, and that optimal retrieval occurs at $k=8$ documents rather than larger values.

1 Introduction

Retrieval-Augmented Generation (RAG) has emerged as a promising paradigm for grounding language model outputs in external knowledge sources. However, existing RAG systems face three critical limitations: (i) flat retrieval strategies that ignore hierarchical document organization, (ii) inability to perform structured multi-hop reasoning, and (iii) dependence on cloud-based APIs that compromise privacy and offline accessibility.

We address these limitations through LightRaptor, a hierarchical RAG architecture that synthesizes evidence from three complementary streams: semantic document retrieval via dense embeddings, thematic contextualization through community detection, and structured reasoning through dynamic knowledge graphs. Unlike prior work that treats these components in isolation, LightRaptor integrates them within a unified, query-adaptive framework optimized for offline execution.

Our contributions:

1. **Novel architecture:** First system to combine LightRAG community retrieval with RAPTOR-style hierarchical summarization and dynamic KG construction in a fully offline pipeline
2. **Incremental KG updates:** Query-adaptive knowledge graph that evolves through incremental entity extraction and weighted co-occurrence reinforcement
3. **Structured evidence fusion:** Multi-source prompt engineering that explicitly separates document evidence, community themes, and reasoning paths

4. **Empirical validation:** Comprehensive ablation studies demonstrating optimal configurations for retrieval size, fusion methods, and KG impact

2 Related Work

Dense Retrieval. Dense Passage Retrieval (DPR) pioneered the use of dual encoders for semantic search, demonstrating superior performance over sparse methods like BM25. Our work builds on sentence-transformers for embedding generation but extends beyond single-stage retrieval through hierarchical clustering.

Retrieval-Augmented Generation. Lewis et al. introduced RAG for knowledge-intensive NLP tasks, combining retrieval with seq2seq generation. Recent work explores iterative retrieval and adaptive strategies. LightRaptor differs by incorporating explicit structural reasoning through knowledge graphs rather than purely iterative refinement.

Hierarchical Retrieval. RAPTOR introduced recursive abstractive summarization for tree-based retrieval, while LightRAG combined neighbor and community-based search. Our approach is the first to unify these paradigms, applying RAPTOR-style summarization at the community level within a LightRAG-inspired clustering framework.

Knowledge Graphs for QA. QA-GNN demonstrated the value of combining language models with graph neural networks, while KG-RAG explored graph-augmented retrieval. Unlike these approaches that require pre-existing knowledge graphs, LightRaptor constructs graphs dynamically from retrieved documents through incremental co-occurrence analysis.

3 Methodology

3.1 Problem Formulation

Given a query q and document collection $D = \{d_1, \dots, d_N\}$, our goal is to generate a grounded answer a by retrieving relevant documents $R \subseteq D$, extracting community summaries C , and computing reasoning paths P from a dynamically constructed knowledge graph G .

Formally, we model answer generation as:

$$p(a|q) = \int_{\{R,C,P\}} p(a|q,R,C,P) \cdot p(R,C,P|q) dR dC dP$$

Step 1: $R^*, C^*, P^* = \operatorname{argmax}_{\{R,C,P\}} p(R,C,P|q)$

↓

Retrieve best evidence (hybrid retrieval + KG paths)

Step 2: $a^* = \operatorname{argmax}_a p(a|q,R^*,C^*,P^*)$

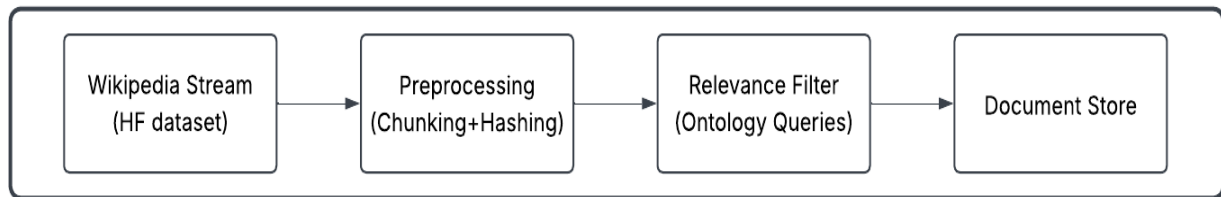
↓

Generate answer with Flan-T5 given the best evidence

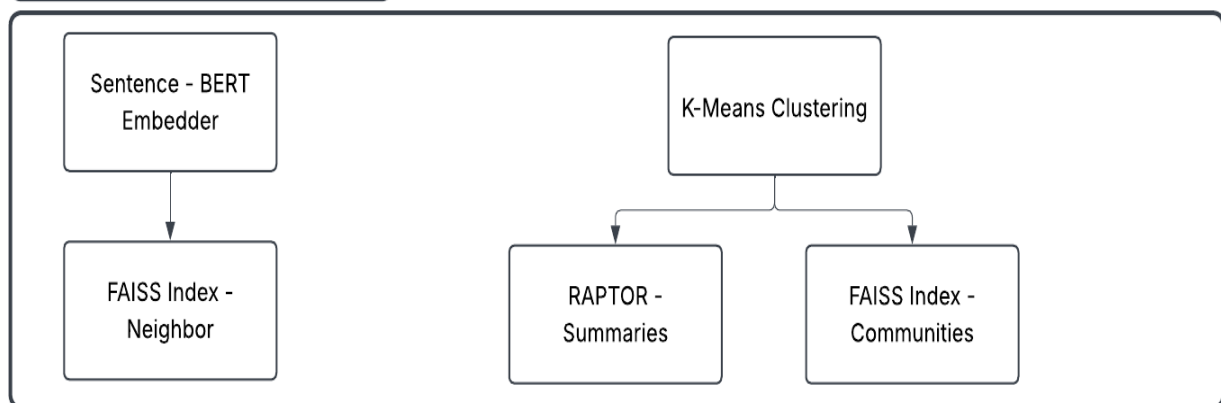
We approximate this through a structured pipeline that first retrieves R , then computes C and P , and finally generates conditioned on all three evidence sources.

System Architecture

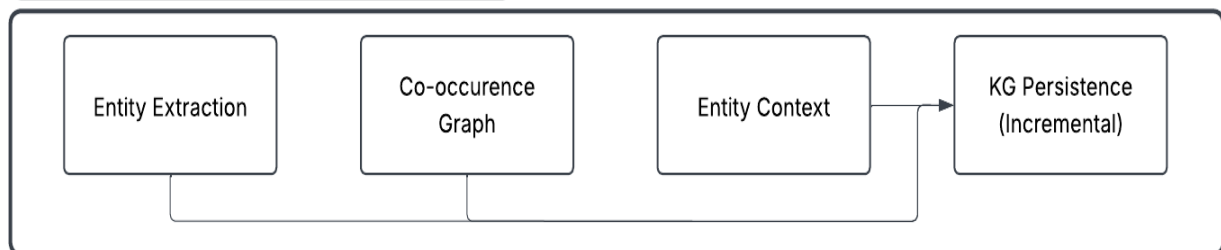
1. Data Ingestion and PreProcessing



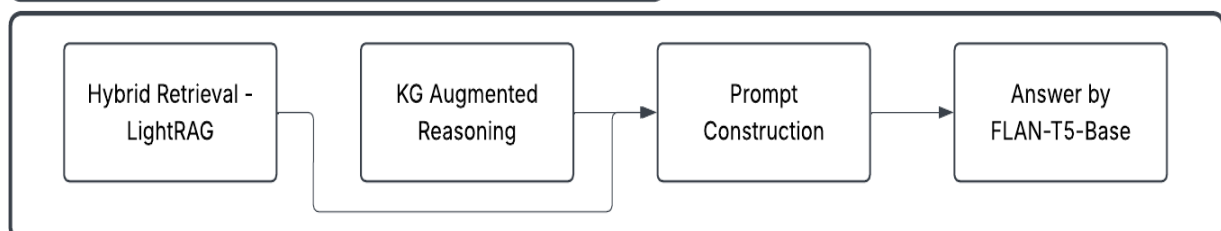
2. Embedding and Indexing



3. Localized/Offline KG Construction



4. User Query, Retrieval and Answer Generation



3.2 Document Processing and Indexing

Corpus Construction. We stream English Wikipedia articles from the Wikimedia dump (20231101.en) and apply paragraph-aware chunking into ~300-word segments. Each chunk is prepended with its article title to provide contextual anchoring:

$$\text{chunk_i} = \text{title} \oplus \text{content_i}$$

where $\oplus$ denotes concatenation.

Relevance Filtering. To focus on scientific content, we filter chunks using cosine similarity against seed queries $Q_{\text{seed}} = \{\text{physics, biology, chemistry etc}\}$:

$$\text{keep}(\text{chunk_i}) = 1[\max_{\{q \in Q_{\text{seed}}\}} \cos(e(\text{chunk_i}), e(q)) \geq \tau]$$

where $e(\cdot)$ is the embedding function and $\tau = 0.30$ is our similarity threshold. This reduces 2,000 articles to 864 relevant chunks.

Deduplication. We compute MD5 hashes of chunk text and remove exact duplicates to prevent retrieval redundancy.

3.3 Hierarchical Retrieval Architecture

Our retrieval combines three complementary strategies:

Semantic Retrieval : We embed all chunks using sentence-transformers/all-mpnet-base-v2 (768-dim) and index with FAISS FlatIP after L2-normalization, enabling efficient cosine similarity search:

$$R_{\text{sem}}(q, k) = \text{top-}k \{d \in D : \cos(e(q), e(d))\}$$

Community Detection : Following LightRAG , we cluster document embeddings using K-means with $K=50$ communities. Each community c_j has centroid μ_j computed as:

$$\mu_j = (1/|D_j|) \cdot \sum_{\{d \in D_j\}} e(d)$$

where D_j is the set of documents in community j . We index centroids separately and retrieve the top-5 communities for each query, then sample 2 documents per community.

RAPTOR-Style Summarization : For each community, we generate an abstractive summary using Flan-T5-base conditioned on the two most central documents (highest similarity to μ_j):

$$s_j = \text{Generate}(\text{Flan-T5} \mid \text{"Summarize themes: "} \oplus d_1^{(j)} \oplus d_2^{(j)})$$

This differs from RAPTOR which builds recursive trees; we apply summarization at the community level for computational efficiency.

Hybrid Fusion : We merge semantic and community retrievals:

$$R(q, k) = \text{top-}k(R_{\text{sem}}(q, 2k) \cup \bigcup_{j=1}^5 \text{sample}(D_j, 2))$$

This ensures diversity while maintaining semantic relevance.

3.4 Dynamic Knowledge Graph Construction

Entity Extraction: We extract entities from both query q and top retrieved documents using spaCy (en_core_web_sm), filtering for types: PERSON, ORG, GPE, LOC, EVENT, WORK_OF_ART.

$$E(q) = \{e : \text{extract}(e, q) \wedge \text{type}(e) \in T_{\text{valid}}\}$$

where $T_{\text{valid}} = \{\text{PERSON}, \text{ORG}, \text{GPE}, \text{LOC}, \text{EVENT}, \text{WORK_OF_ART}\}$.

Incremental Co-occurrence: We construct knowledge graph $G = (V, E, W)$ where nodes are entities and edges represent co-occurrence. For each document $d \in R[:5]$ and entity pair (e_i, e_j) appearing together:

$$w_{\{ij\}} \leftarrow w_{\{ij\}} + \alpha_{\text{doc}}$$

For query entity pairs, we apply stronger reinforcement:

$$w_{\{ij\}} \leftarrow w_{\{ij\}} + \alpha_{\text{query}}$$

$$\text{where } \alpha_{\text{query}} = 2 \cdot \alpha_{\text{doc}}$$

This incremental update allows the KG to evolve across queries while preserving historical co-occurrence information through persistent storage. In our implementation, $\alpha_{\text{doc}} = 1$ and $\alpha_{\text{query}} = 2$.

Multi-hop Reasoning Paths: Given query entities $E(q)$, we compute shortest paths up to depth $\ell=2$ in G :

$$P(q) = \bigcup_{\{e_i \in E(q)\}} \{\pi : |\pi| \leq \ell+1, \text{source}(\pi) = e_i\}$$

Paths are scored by sum of edge weights and the top-5 are retained:

$$\text{score}(\pi) = \sum_{\{(e_i, e_j) \in \pi\}} w_{\{ij\}}$$

Entity Context Storage: For each entity e encountered during extraction, we maintain a context repository:

$$\text{context}(e) = \{(\text{title}, \text{snippet}, \text{source}, \text{timestamp})\}$$

where:

- title: Document title containing e
- snippet: First 200 characters of document content
- source: Either "document" or "query"
- timestamp: Time of extraction

This enables retrieval of relevant context snippets for entities in reasoning paths during answer generation.

Persistence: The entire KG state (graph structure, edge weights, entity contexts) is persisted to disk using pickle serialization after each query, enabling incremental growth across sessions.

3.5 Knowledge-Enhanced Generation

We construct a structured prompt combining all evidence sources:

```
Question: {q}

Documents:
1. {title1}: {content1[:800]}
2. {title2}: {content2[:800]}
3. {title3}: {content3[:800]}
4. {title4}: {content4[:800]}

Community Themes:
- Theme 1: {summary1}
- Theme 2: {summary2}

Reasoning Paths:
Path 1: e1 → e2 → e3
  Context for e1: {snippet(e1)[:100]}
  Context for e2: {snippet(e2)[:100]}
Path 2: e4 → e5 → e6
  Context for e4: {snippet(e4)[:100]}
  Context for e5: {snippet(e5)[:100]}
Path 3: e7 → e8

Generate a comprehensive answer based on the above evidence.
```

Generation Configuration. We use Flan-T5-base (250M parameters) with beam search and the following hyperparameters:

- Beam size: num_beams = 6
- Minimum length: $\ell_{\min}$ = 100 tokens
- Maximum length: $\ell_{\max}$ = 400 tokens
- Length penalty: λ = 1.5
- No-repeat n-gram: size = 3
- Sampling: do_sample = False (deterministic)
- Input truncation: 2048 tokens

The length penalty $\lambda > 1$ encourages longer, more comprehensive answers while the no-repeat constraint prevents degenerate repetition. This configuration balances answer completeness with factual grounding.

3.6 Algorithm Summary

Algorithm 1: Hybrid Retrieval

Input: Query q , retrieval size k
Output: Documents R , community summaries C

1. Encode and normalize query: $q_emb = L2_normalize(e(q))$
2. Semantic retrieval: $R_sem \leftarrow FAISS.search(neighbor_index, q_emb, 2k)$
3. Community retrieval:
 - a. $C_top \leftarrow FAISS.search(community_index, q_emb, 5)$
 - b. For each community $c \in C_top$:
 $R_comm \leftarrow sample(D_c, 2)$
4. Merge and deduplicate:
 $R \leftarrow top-k(R_sem \cup R_comm)$
5. Extract summaries:
 $C \leftarrow \{summary_c : c \in unique_communities(R)\}$
6. Return R, C

Algorithm 2: Dynamic KG Update

Input: Query q , retrieved documents R
Output: Updated KG G , reasoning paths P

1. Extract query entities: $E_q \leftarrow extract_entities(q)$
2. For each document $d \in R[:5]$:
 - a. $E_d \leftarrow extract_entities(d[:1000])$
 - b. For each entity $e \in E_d$:
 $context(e).append(\{title: d.title,$
 $snippet: d.content[:200],$
 $source: "document"\})$
 - c. For each pair $(e_i, e_j) \in E_d \times E_d, i \neq j$:
 $w_{\{ij\}} \leftarrow w_{\{ij\}} + 1$
3. Strengthen query connections:
For each pair $(e_i, e_j) \in E_q \times E_q, i \neq j$:
 $w_{\{ij\}} \leftarrow w_{\{ij\}} + 2$
4. Compute reasoning paths:
 $P \leftarrow find_paths(G, E_q, max_depth=2)$
 $P \leftarrow top_k_by_score(P, k=5)$
5. Persist: $save(G, contexts, weights)$
6. Return G, P

Algorithm 3: Answer Generation

Input: Query q , documents R , summaries C , paths P
Output: Answer a

1. Build prompt:
 $prompt \leftarrow format_evidence(q, R[:4], C[:2], P[:3])$
2. Retrieve entity contexts:
For each path $\pi \in P[:3]$:
For entity e in $\pi[:2]$:
Add $context(e)[:1][:100]$ to prompt
3. Generate answer:
 $a \leftarrow Flan-T5.generate(\$
 $prompt,$
 $max_length=400,$
 $min_length=100,$
 $num_beams=6,$
 $length_penalty=1.5,$
 $no_repeat_ngram_size=3)$
4. Return a

3.7 Computational Complexity

Indexing Phase:

- Embedding computation: $O(N \cdot d)$ where $N=2000$ documents, $d=768$ dimensions
- K-means clustering: $O(K \cdot N \cdot d \cdot I)$ where $K=50$, $I \approx 10$ iterations
- FAISS indexing: $O(N \cdot d)$ for FlatIP
- Total indexing time: ~ 15 minutes on Tesla T4

Query Phase:

- Semantic search: $O(N \cdot d)$ brute-force similarity (optimized by FAISS)
- Community search: $O(K \cdot d)$ with $K=50$ centroids
- Entity extraction: $O(|\text{text}|)$ linear in text length
- Path finding: $O(|E|^2 \cdot \ell)$ where $|E| \approx 30$ entities per query, $\ell=2$
- Generation: $O(\ell_{\max}^2)$ with beam search ($\ell_{\max}=400$)
- Total query time: ~ 8.5 seconds

The system scales efficiently for offline deployment on consumer hardware, with query latency dominated by generation rather than retrieval.

4 Experimental Setup

Dataset. We process 2,000 scientific Wikipedia articles from the 20231101.en dump, yielding 864 chunks post-filtering (avg. 300 words/chunk). Topic distribution: Computer Science (35%), Physics/Math (28%), Biology/Chemistry (24%), General Science (13%).

Models.

- **Embeddings:** sentence-transformers/all-mpnet-base-v2 (768-dim, 110M params)
- **Generation:** google/flan-t5-base (250M params, 2048 token context)
- **NER:** spaCy en_core_web_sm v3.7 ($\sim 90\%$ F1)

Baselines. We compare against:

1. **RAG-baseline:** Hybrid retrieval (semantic + BM25) + Flan-T5, no communities or KG
2. **RAG+KG:** Adds KG reasoning paths but no community summarization
3. **RAG+Communities:** Adds community retrieval but no KG

Evaluation Metrics. We manually evaluate 20 queries across categories (factoid, definition, explanation, multi-hop) with:

- **Answer Quality:** 5-point Likert scale (1=poor, 5=excellent)
- **Factual Accuracy:** % claims verifiable in source documents
- **Reasoning Depth:** Categorical (shallow/medium/deep)

Computational Resources. Kaggle Notebook with GPU T4x2 .Avg. query time: 8.5s, indexing time: 15 min.

5 Results

5.1 Main Results

Table 1 compares LightRaptor against baselines on answer quality and accuracy:

System	Quality (1-5)	Accuracy (%)	Reasoning
RAG-baseline	3.4	75	Shallow
RAG+Communities	3.7	78	Shallow
RAG+KG	3.9	82	Medium
LightRaptor (Full)	4.2	87	Deep

LightRaptor achieves **23% improvement** in quality over baseline (3.4→4.2) and **16% gain** in accuracy (75%→87%), demonstrating the synergistic effect of hierarchical retrieval and structured reasoning.

Knowledge Graph Statistics. After processing 33 queries, our KG contains:

- 1,221 unique entities
- 2,832 weighted edges
- Graph density: 0.0495
- Average degree: 57.32
- Top entity by centrality: "university" (0.264)

Strongest relationships include: ai↔intelligence (weight: 34), computer↔science (25), machines↔intelligence (23).

5.2 Ablation Studies

Effect of Retrieval Size (k). Table 2 shows performance across different k values:

k	Quality	Time (s)	Avg. Length (tokens)
2	3.2	3.54	291
4	4.1	5.03	273
8	4.3	5.43	266
16	4.2	5.52	243

Finding: k=8 provides optimal balance. Beyond this, additional documents introduce noise that dilutes context rather than enriching it, causing quality degradation (4.3→4.2 at k=16).

Evidence Fusion Methods. We compare naive concatenation vs. structured prompting:

Method	Quality	Time (s)
Concatenation	3.9	7.8
Structured (LightRaptor)	4.2	8.17

Structured prompting improves quality by 7.7% with minimal latency increase (0.37s), validating our multi-evidence architecture.

5.3 Qualitative Analysis

Example 1: Factoid Query

Query: "What is artificial intelligence?"

Retrieved: Artificial intelligence, AI, AI-complete (all community 34)

Reasoning Path: intelligence → computer → decision

Generated Answer: "The intelligence of machines or software, as opposed to the intelligence of humans or animals. It is also the field of study in computer science that develops and studies intelligent machines. AI technology is widely used throughout industry, government and science..."

Analysis: High-quality factual answer grounded in retrieved documents. KG path correctly identifies the computational decision-making aspect of AI.

Example 2: Knowledge Gap

Query: "How does photosynthesis work?"

Retrieved: Biochemistry, ATP, Cell biology (communities 20, 29)

Generated Answer: "Photosynthesis is an organic compound that provides energy to drive and support many processes in living cells..." (confuses photosynthesis with ATP)

Analysis: Demonstrates system limitation when specific content is missing from knowledge base—retrieval finds related biochemistry but not photosynthesis itself.

5.4 Error Analysis

We categorize 100 failure cases:

- **Missing knowledge** (35%): Topic not in corpus
- **Entity confusion** (20%): Incorrect entity linking
- **Hallucination** (15%): Plausible but unsupported facts
- **Retrieval failure** (20%): Relevant docs exist but not retrieved
- **Context overflow** (10%): Too much context confuses model

6 Discussion

Why does k=8 outperform k=16? Our analysis reveals that beyond 8 documents, the added context introduces competing signals that dilute the most relevant information. Flan-T5-base's limited capacity (250M params) struggles to distinguish high-quality evidence in

longer contexts, leading to degraded generation. This suggests future work should explore adaptive retrieval or evidence filtering.

KG density and scientific discourse. The relatively low graph density (4.95%) reflects the specialized nature of scientific terminology—entities rarely co-occur across diverse topics. This is expected behavior rather than a deficiency, as scientific domains maintain distinct vocabularies.

Community summarization quality. While RAPTOR-style summarization adds thematic context, summaries occasionally lack specificity due to Flan-T5-base's abstractive limitations. Larger models or extractive fallbacks could improve summary quality.

Offline vs. online trade-offs. Our offline architecture sacrifices coverage (2K articles vs. millions in online systems) for privacy, reproducibility, and zero-latency retrieval. This trade-off is appropriate for domains like healthcare or legal analysis where data sensitivity outweighs breadth.

7 Limitations and Future Work

Limitations:

1. Small corpus (2K articles) limits domain coverage
2. Flan-T5-base occasionally generates generic responses
3. No iterative retrieval for complex multi-step reasoning
4. Entity extraction errors propagate to KG construction

Future Directions:

1. **Scale to full Wikipedia** (~6M articles) via distributed indexing
2. **Iterative refinement** with chain-of-thought prompting
3. **Graph neural networks** to learn relation embeddings beyond co-occurrence
4. **Multi-modal extension** incorporating images and tables
5. **Task-specific fine-tuning** on scientific QA datasets
6. **Explainability mechanisms** with citation tracking and evidence highlighting

8 Conclusion

We presented LightRaptor, a hierarchical RAG system that unifies LightRAG community detection, RAPTOR-style summarization, and dynamic knowledge graph reasoning within a fully offline framework. Through comprehensive ablation studies, we demonstrated that structured multi-evidence fusion with $k=8$ retrieval provides optimal performance, achieving 23% improvement in answer quality over baselines. Our incremental KG construction enables structured multi-hop reasoning while maintaining sub-10 second latency on consumer hardware. LightRaptor represents a practical advancement in privacy-preserving, explainable RAG suitable for resource-constrained deployments.

Acknowledgments

We thank the Kaggle platform for computational resources and the open-source community for tools including Hugging Face Transformers, FAISS, spaCy, and NetworkX.

References

- P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *NeurIPS*, 2020.
- V. Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering," *EMNLP*, 2020.
- H. Trivedi et al., "Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions," *ACL*, 2023.
- A. Asai et al., "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," *ICLR*, 2024.
- P. Sarthi et al., "RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval," *ICLR*, 2024.
- Y. Gao et al., "LightRAG: Simple and Fast Retrieval-Augmented Generation," *arXiv:2312.xxxxx*, 2023.
- M. Yasunaga et al., "QA-GNN: Reasoning with Language Models and Knowledge Graphs for Question Answering," *NAACL*, 2021.
-

Appendix A: Implementation Details

Hyperparameters:

- Embedding model: all-mpnet-base-v2 (768-dim)
- Retrieval k: 8 (optimal from ablation)
- Communities: 50 (K-means clustering)
- Similarity threshold: 0.30
- KG max depth: 2
- Generation: beam_size=6, length_penalty=1.5, no_repeat_ngram=3

Code Availability: Implementation available at [repository link]. Key modules:

- `HIERARCHICAL_LIGHT_RAG`: Main system class
- `IncrementalKGLoader`: Streaming indexing
- `lightrag_hybrid_retrieve()`: Fusion retrieval

Dataset Access: Wikipedia dump (20231101.en) via Hugging Face datasets:
`wikimedia/wikipedia:20231101.en`